Introduction to Parallel Algorithms and Programming

Sessions 2 and 4: Parallel algorithms in the PRAM model

Oguz Kaya

Assistant Professor
Université Paris-Saclay and LISN, Gif-sur-Yvette, France

université PARIS-SACLAY

Objectives

- Revise sequential algorithms, the **RAM** model, and complexity analysis
- Introduce the **PRAM** model of parallel computation and its variants: EREW, CREW, CRCW
- Learn what a PU knows, and what a valid PRAM algorithm looks like on each variant
- Design and analyse first PRAM algorithms: array multiplication, search, broadcast, reduction
- Learn the **pointer jumping** technique on linked lists: list ranking
- Learn the metrics for the complexity and performance of PRAM algorithms: time, work, cost, speedup, efficiency
- Apply them to every algorithm of the session: which ones are work-optimal, which ones are efficient?

Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

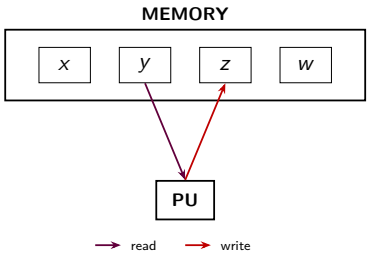
Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

The RAM model

The RAM (random access machine) model is an abstraction of a sequential computer.

- Consists of a processing unit (PU) and an associated memory space.
- Access to each element in the memory is done in constant time.
- Each operation is performed in constant time in the PU.
- Equivalent to a Turing machine (simulation); belongs to the class of "register machines".
- Not very realistic (no L1/L2/L3/DRAM memory hierarchy), yet useful for developing algorithms that are **asymptotically optimal**.
  - One should then adapt these algorithms to modern computer architectures (i.e., multicore CPUs, GPUs, etc., the focus of HPC).



Algorithms and complexity

An algorithm is a sequence of operations on a RAM machine, which aims to solve a given problem.

- Each memory access and arithmetic operation takes unit time.
- The **time complexity** of the algorithm is a function $T(N)$ of the problem size, which represents the total number of operations it performs.
- One can also consider its **space complexity**, the total memory space it uses in execution.

SumArray$(A[1 \dots N])$

1: $sum \leftarrow 0$
2: **for** $i = 1, \dots, N$ **do**
3: $\quad sum \leftarrow sum + A[i]$
4: **return** sum

Complexity analysis of SumArray

Count the elementary operations (memory accesses, arithmetic, comparisons), each taking unit time.

- Line 1: one write, 1 operation.
- Lines 2–3: N iterations; each iteration reads $A[i]$ and *sum*, adds, writes *sum*, increments and tests i: a constant number c of operations.
- $T(N) = c\,N + O(1) = O(N)$: **linear** time.
- The exact constant c depends on the machine and the compiler; only the **order of growth** $O(N)$ is meaningful, algorithmically speaking.
- Space complexity: $O(N)$. The input itself must be stored; we cannot ignore the input size.

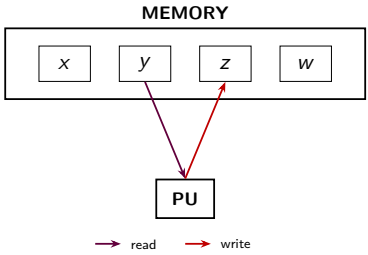
SumArray$(A[1 \ldots N])$

1: $sum \leftarrow 0$
2: **for** $i = 1, \ldots, N$ **do**
3: $\quad sum \leftarrow sum + A[i]$
4: **return** sum

Complexity analysis

Many methods exist for complexity analysis:

- Nested loops: counting techniques in the loop iteration domain
- Recursion / divide-and-conquer: counting by hand, induction, Master theorem
- Probabilistic analysis (e.g., quicksort)
- Amortized analysis (e.g., dynamic tables)
- These analyses can be performed for
  - the best case
  - the worst case
  - the average case



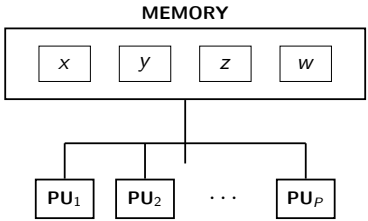
Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

The PRAM model

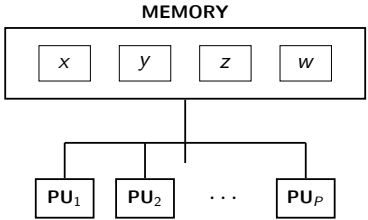
The parallel random access machine (PRAM) is an abstraction of a parallel computer.

- Consists of P PUs and a memory space shared among all PUs.
- PUs
  - execute the same algorithm/code in a **synchronous manner**, but on different elements in the memory (i.e., à la SIMD): all of them attempt to execute the **same instruction** of the code in the **same clock cycle**;
  - more akin to a synchronous **SIMT** execution (rather than SIMD, which operates on contiguous data elements), as in GPUs (which are only partially synchronous);
  - can perform an arithmetic operation, or read from / write to different memory locations, **simultaneously** in constant time;
  - can have **private memory/registers** for PU-specific information (identifier, program counter) and temporary results (e.g., data loaded from the shared memory before an arithmetic operation).



The PRAM model (cont.)

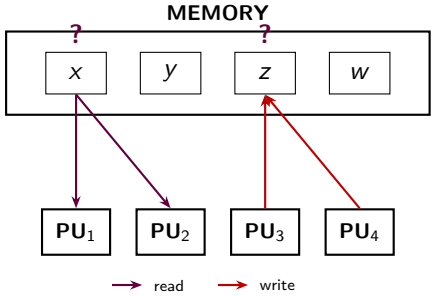
- Read/write on the same memory location?
  - This behaviour defines the different PRAM variants.
- Number of PUs: potentially infinite!
- Memory size: potentially infinite!
- Communication cost: completely ignored (done implicitly through the fast shared memory)
  - PRAM is useful for developing **asymptotically optimal** parallel algorithms.
  - One should then adapt these algorithms to modern parallel computer architectures using HPC techniques.



Memory conflicts in PRAM

During the execution of a PRAM algorithm, multiple PUs could try to access the same memory location (memory conflict).

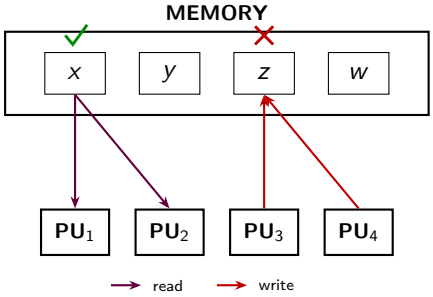
- Possible scenarios?
  - **Concurrent reads** of the same memory location by multiple PUs.
  - **Concurrent writes** to the same memory location by multiple PUs.
- Each of them is either allowed or forbidden by the model.
- This yields four PRAM variants: **CREW**, **CRCW**, **EREW**, and ERCW.



PU_1, PU_2 read x at the same time; PU_3, PU_4 write z at the same time. Allowed?

The CREW PRAM model

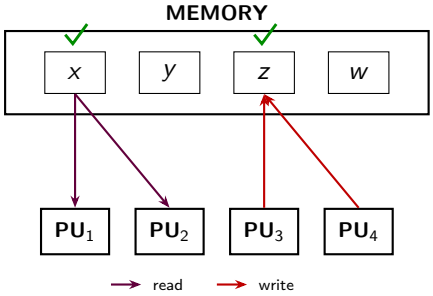
- **CREW: Concurrent Read, Exclusive Write**
  - Simultaneous reading of the same memory location is allowed, and takes constant time.
  - Simultaneous writing is forbidden (bug).
  - The standard PRAM model in the literature.



Concurrent read of x: allowed. Concurrent write of z: **forbidden**.

The CRCW PRAM model

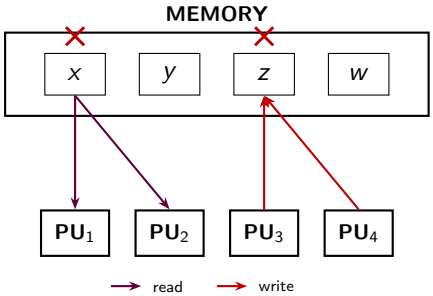
- **CRCW: Concurrent Read, Concurrent Write**
  - Simultaneous reading/writing from/to the same memory location is allowed, and takes constant time.
  - The most powerful model.
  - A "concurrency mode" is employed for simultaneous writes:
    - **Consistent** mode: all PUs write the same value.
    - **Arbitrary** mode: one PU's value is retained, chosen arbitrarily.
    - **Priority** mode: the value of the PU with the lowest index is retained.
    - **Fusion** mode: a commutative and associative operation is applied ($+$, $\times$, min, max, and, or, xor, ...).
  - Sometimes the mode is imposed by the problem; other times we have the liberty to choose the mode to employ.
  - Not very realistic: real hardware has a hard time handling simultaneous write requests to the same memory location (mechanisms exist, e.g., atomic operations, but they are quite expensive).



Concurrent read of x: allowed. Concurrent write of z: allowed, resolved by the concurrency mode.

The EREW PRAM model

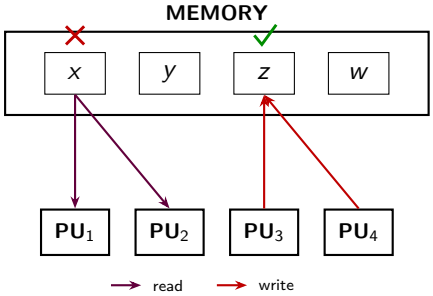
- **EREW: Exclusive Read, Exclusive Write**
  - Simultaneous read/write of the same memory location by multiple PUs is forbidden.
  - The most restrictive model.



Concurrent read of x: **forbidden**. Concurrent write of z: **forbidden**. Only accesses to different cells may happen at the same time.

The ERCW PRAM model

- **ERCW: Exclusive Read, Concurrent Write**
  - Does not exist. Why?
    - Does not make sense architecturally: hardware that can merge concurrent writes can serve concurrent reads.



Concurrent read of x: **forbidden**, but concurrent write of z: allowed?!

Which model is the most realistic: CREW or EREW?

Both are, depending on the **scope** ("granularity") of the hardware we look at.

Small scope: CREW-like behaviour

- **SIMD lanes**: one `set(val, vector)` instruction copies `val` into all 8 lanes of `vector` in one cycle.
- **Cores of one CPU**: each core keeps a copy of what it reads in its cache; once loaded, many cores read the same value for free.
- **GPU threads**: reading the same address, the hardware fetches it once and hands it to all of them.
- Concurrent *writes* to one address are always serialized: exclusive write.

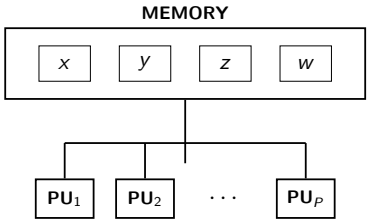
Large scope: EREW-like behaviour

- The *first* time many cores need a value stored far away, the requests queue up and are served one by one.
- **Machines of a cluster**: a value lives in one machine's memory; any other machine needing it must receive a message. Reading is a communication; sending one value to P machines takes $\log_2 P$ rounds.

No need to pick a side: the smaller the scope of the hardware, the more CREW-like its behaviour; the larger the scope, the more EREW-like.

Writing PRAM algorithms: what does a PU know?

- In the **private space** (registers) of each PU:
  - its index i, and the number of active PUs in the parallel region, say N;
  - a **copy of the program**: reading an instruction never causes a read conflict, even on an EREW PRAM;
  - the **array pointers** (addresses of A, *next*, ...) and the constants of the program, considered part of the program: each PU has its own private copy;
  - temporary values, handled transparently by the registers (we will not write them):
    - a read of a shared memory cell is a **load**: it puts the value into a private register;
    - the arithmetic operation is then performed on the **private registers**;
    - an assignment to a shared memory cell is a **store**: the value of a private register is written into the cell.
- The **real data** lives in the **shared memory**: $A[1 \dots N]$, $next[1 \dots N]$, ...
  - All PUs, each with its own index i, accessing the element $A[i]$ at the same time = accessing the cells (address of A) + i: different PUs, different cells, **exclusive** access.
  - Several PUs reading the *same* cell in the same step: **concurrent read**. Fine on CREW/CRCW, forbidden on EREW.

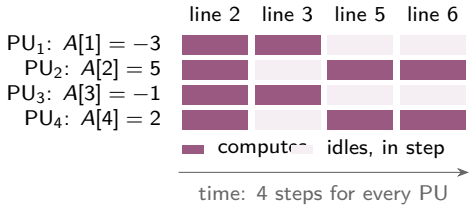


Writing PRAM algorithms: synchronous execution and if

- Lockstep: all PUs execute the **same instruction at the same time**. And an **if**?
- Line 2: all PUs evaluate the condition together (one step). Line 3: all PUs go through the **then** branch together, but only those whose condition is true **really compute**; the others **idle**, in step.
- Lines 5–6: the **else** branch, by all PUs together again, only the *other* PUs computing. The branches run **one after the other**.
- Time of the **if**, for every PU: test + **then** + **else** $= 1 + 1 + 2 = 4$ steps here, even for PU_1: the **sum** of the branches, not the longest.
- Same on GPUs (lanes masked off by the hardware) and SIMD units (no lane skips anything: compute both sides, select per lane).

Example: if/else in a forall

1: **forall** $i = 1, \ldots, N$ **in parallel do**
2: **if** $A[i] < 0$ **then**
3: $A[i] \leftarrow 0$
4: **else**
5: $A[i] \leftarrow A[i] \cdot A[i]$
6: $A[i] \leftarrow A[i] + 1$



Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

A first PRAM algorithm: multiplying an array by 2

Given an array $A[1 \dots N]$, multiply each element by 2.

- Sequential algorithm/complexity?
  - One loop over the array: $O(N)$.
- PRAM algorithm: one PU per element, N PUs. Line 2 = three elementary steps for PU i:
  - $temp \leftarrow \text{LOAD}(A + i)$: the value $A[i]$ goes into a private register;
  - $temp \leftarrow 2 \cdot temp$: arithmetic on the register (2 is a constant of the program);
  - $\text{STORE}(A + i, temp)$: the register is written back into $A[i]$.
  - Done by all PUs **simultaneously**: parallel time complexity $T(N) = O(1)$, whatever N!
- Which PRAM models can run it?
  - Reads: PU i reads only cell $A[i]$: **exclusive**.
  - Writes: PU i writes only cell $A[i]$: **exclusive**.
  - Runs on an **EREW** PRAM, hence also on CREW and CRCW.

MULTIPLYBYTWO($A[1 \dots N]$)

1: **forall** $i = 1, \dots, N$ **in parallel do**
2: $\quad A[i] \leftarrow 2 \cdot A[i]$

Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

Another PRAM algorithm: search in an array

Given an array $A[1 \dots N]$ and a key e, find an index i such that $A[i] = e$.

- Sequential: $O(N)$. PRAM: $O(1)$ with N PUs, all comparisons at once.
- Which PRAM models can run it? **Reads** first (line 3): $A[i]$ is exclusive, no problem; e is read by all PUs at the same time: **not exclusive**, EREW does not work.

SEARCHINDEX$(A[1 \dots N], e)$

1: $idx \leftarrow 0$
2: **forall** $i = 1, \dots, N$ **in parallel do**
3: **if** $A[i] = e$ **then**
4: $idx \leftarrow i$
5: **return** idx

- **Writes**: $idx \leftarrow 0$ (line 1) is in the sequential part; $idx \leftarrow i$ (line 4) is in the parallel part. Can this run on a CREW PRAM? Nuanced: if the elements of A are **unique**, at most one PU writes idx: valid CREW algorithm. Otherwise, several PUs may write idx simultaneously: **CRCW** required.
- CRCW: which concurrency modes would work?
  - Consistent? No: different values i are written.
  - Arbitrary? Yes: one of the matching indices.
  - Priority (lowest PU index)? Yes: the first occurrence.
  - Fusion with $+$? No: a sum of indices. With min/max? Yes: the first/last occurrence.

Search on an EREW PRAM: how?

The only obstacle for EREW: all PUs read e at the same time (line 3).

- Solutions?
  - Give each PU its **own copy** of e, in the shared memory: an array $pe[1 \dots N]$ with $pe[i] = e$.
  - Line 3 becomes **if** $A[i] = pe[i]$: PU i reads only $pe[i]$, an **exclusive** read.
- How to produce $pe[1 \dots N]$ from e, without any concurrent read?
  - Making N copies of one value is the **broadcast** operation.

SEARCHINDEX$(A[1 \dots N], e)$

1: $idx \leftarrow 0$
2: **forall** $i = 1, \dots, N$ **in parallel do**
3: **if** $A[i] = e$ **then**
4: $idx \leftarrow i$
5: **return** idx

Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

Broadcast on an EREW PRAM

Given a value e, produce an array $pe[1 \dots N]$ with $pe[i] = e$ for all i, without any concurrent read.

- Naive: PU 1 copies e into $pe[1], \dots, pe[N]$ one by one: $O(N)$, no better than sequential!
- **Doubling**: every PU that already has the value passes it on to one PU that does not: $1, 2, 4, 8, \dots$ copies.
- Step k: PU $i \leq 2^k$ reads $pe[i]$ and writes $pe[i + 2^k]$: all accesses are **exclusive**.
- Parallel time: $\lceil \log_2 N \rceil$ iterations of an $O(1)$ body: $T(N) = O(\log N)$ with N PUs.
- The fundamental EREW building block: needed for any value not in the program.
- Note: N PUs are created at every step, but only 2^k of them copy; BCAST **can be made work-optimal** (see its analysis later).

BCAST(e, N)

```
1: pe[1] ← e
2: for k = 0, ..., ⌈log2 N⌉ − 1 do
3:     forall i = 1, ..., N in parallel do
4:         if i ≤ 2^k and i + 2^k ≤ N then
5:             pe[i + 2^k] ← pe[i]
6: return pe[1 ... N]
```

Search on an EREW PRAM

Given an array $A[1 \dots N]$ and a key e, find an index i such that $A[i] = e$ (unique elements).

- First broadcast e into $pe[1 \dots N]$, then PU i compares $A[i]$ with $pe[i]$.
- All accesses are now exclusive: runs on an **EREW** PRAM.
- Complexity with N PUs?
  - Line 2: $O(\log N)$ (BCAST).
  - Lines 3–5: $O(1)$.
  - $T(N) = O(\log N)$, instead of $O(1)$ on a CREW PRAM.
- The whole $\log N$ comes from the broadcast: the price of exclusive reads.

SEARCHINDEXEREW($A[1 \dots N], e$)

1: $idx \leftarrow 0$
2: $pe[1 \dots N] \leftarrow \text{BCAST}(e, N)$
3: **forall** $i = 1, \dots, N$ **in parallel do**
4: **if** $A[i] = pe[i]$ **then**
5: $idx \leftarrow i$
6: **return** idx

Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

Back to SumArray: on a CRCW PRAM

Given an array $A[1 \dots N]$, compute $A[1] + A[2] + \cdots + A[N]$. Sequential: $O(N)$.

- How to do it on a CRCW PRAM with N PUs?
  - All PUs write their element into the *same* cell *sum*: a concurrent write.
  - Choose the **fusion mode with** $+$: the N written values are added, and *sum* receives their total.
  - The mode is *our* choice here: the problem does not impose it.
- Complexity?
  - One read of $A[i]$ and one (concurrent) write: $T(N) = O(1)$ with N PUs.
- The whole work of summing is done by the memory, "for free". Which is exactly why CRCW is not very realistic.

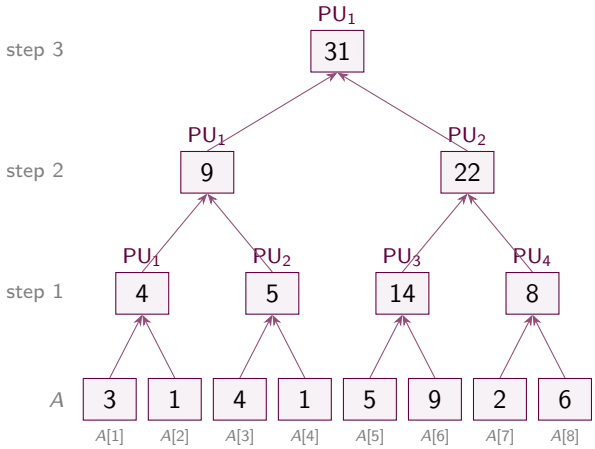
SumArrayCRCW($A[1 \dots N]$)

1: $sum \leftarrow 0$
2: **forall** $i = 1, \dots, N$ **in parallel do**
3: $\quad sum \leftarrow A[i]$ ▶ fusion $+$
4: **return** *sum*

SumArray on a CREW/EREW PRAM

Given an array $A[1 \dots N]$, compute $A[1] + A[2] + \cdots + A[N]$ without any concurrent write.

- No fusion mode: a PU can add only two values at a time.
- Idea: add the elements **two by two**, in parallel: $A[1] + A[2]$, $A[3] + A[4]$, ...
- The array size is **halved** at each step; repeat until one element is left.
- $N = 8$: three steps, creating 4, 2, then 1 PUs.
- In general: $\lceil \log_2 N \rceil$ steps.



SumArray on a CREW/EREW PRAM (cont.)

From the tree to PRAM code: at each step, PU i produces the i-th element of the next level.

- PU i reads $A[2i-1]$ and $A[2i]$ (only $A[2i-1]$ if n is odd), writes $A[i]$: each cell is read by one PU and written by one PU: **EREW**.
  - PU i writes $A[i]$ while PU $\lceil i/2 \rceil$ reads it, in the same line: fine, reads happen before writes.
- Complexity: the loop body (lines 3–8) is $O(1)$; n is halved at each iteration: $\lceil \log_2 N \rceil$ iterations, known in advance (n lives in the private space of each PU, no termination test).
- $T(N) = O(\log N)$ with $\lceil N/2 \rceil$ PUs. The input array is overwritten.

SumArrayEREW($A[1 \dots N]$)

1: $n \leftarrow N$
2: **while** $n > 1$ **do**
3: **forall** $i = 1, \dots, \lceil n/2 \rceil$ **in parallel do**
4: **if** $2i \leq n$ **then**
5: $A[i] \leftarrow A[2i-1] + A[2i]$
6: **else**
7: $A[i] \leftarrow A[2i-1]$
8: $n \leftarrow \lceil n/2 \rceil$
9: **return** $A[1]$

Generalizing: reduction on an EREW PRAM

Given $A[1 \dots N]$ and a binary operation $\otimes$, compute $A[1] \otimes A[2] \otimes \cdots \otimes A[N]$. Same idea, written **recursively**: combine two by two, then reduce the half-size array.

- Which operations $\otimes$? The pairs are combined in a different order than the sequential left-to-right computation: $\otimes$ must be **associative**, so that regrouping does not change the result. Commutativity is *not* needed: adjacent pairs $(2i-1, 2i)$ keep the order (it would be if PU i combined $A[i]$ with $A[i + \lceil n/2 \rceil]$).
- Interesting $\otimes$: $+$, $\times$, min, max, logical/bitwise and, or, xor, gcd, matrix product, ...
- Complexity: $T(n) = T(\lceil n/2 \rceil) + O(1)$, hence $T(N) = O(\log N)$ with $\lceil N/2 \rceil$ PUs.
- **Reduction** is the **mirror image of broadcast**: BCAST turns 1 value into N copies by doubling, REDUCE turns N values into 1 by halving; $\lceil \log_2 N \rceil$ steps each.

REDUCE$(A[1 \dots n], \otimes)$

1: **if** $n = 1$ **then return** $A[1]$
2: **forall** $i = 1, \dots, \lceil n/2 \rceil$ **in parallel do**
3: **if** $2i \leq n$ **then**
4: $A[i] \leftarrow A[2i-1] \otimes A[2i]$
5: **else**
6: $A[i] \leftarrow A[2i-1]$
7: **return** REDUCE$(A[1 \dots \lceil n/2 \rceil], \otimes)$

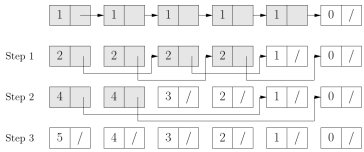
Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

List ranking

Given a linked list $next[1 \dots N]$, find the distance $d[i]$ from each node i to the end of the list.

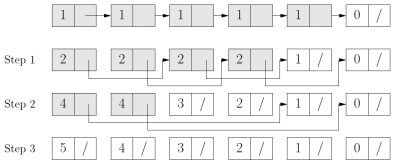
- $next[i] = \text{NIL} \implies d[i] = 0$
- $next[i] = j \neq \text{NIL} \implies d[i] = d[j] + 1$
- Sequential algorithm? Complexity?
  - Recursion + memoisation, in $O(N)$.



List ranking (cont.)

How to parallelize in PRAM?

- Idea: **recursive pointer jumping**
  - First, each pointer jump adds 1 to the distance.
  - Then, as we jump over pointers, we add the distance of the pointer we jump over.
  - Continue until all pointers point to the end of the list.
  - How many iterations until convergence?
    - At each jump, either the jump distance of a pointer doubles, or it points to the end of the list.
    - In $\log_2 N$ iterations, all pointers point to the end of the list, with the correct distance $d[i]$.



List ranking (cont.)

How to write the PRAM algorithm?

- Initialization (lines 1–2) is done in $O(1)$.
- The main loop terminates after $\log_2 N$ iterations, using N PUs.

RankComputation($next[1 \dots N]$)

1: **forall** $i = 1, \dots, N$ **in parallel do**
2: **if** $next[i] = \text{Nil}$ **then** $d[i] \leftarrow 0$ **else** $d[i] \leftarrow 1$
3: **while** there exists a node i such that $next[i] \neq \text{Nil}$ **do**
4: **forall** $i = 1, \dots, N$ **in parallel do**
5: **if** $next[i] \neq \text{Nil}$ **then**
6: $d[i] \leftarrow d[i] + d[next[i]]$
7: $next[i] \leftarrow next[next[i]]$

List ranking (cont.)

Read/write conflicts in lines 6–7?

- Line 6 = elementary steps for PU i ($temp1, \ldots, temp4$: private registers):
  - $temp1 \leftarrow \text{LOAD}(next + i)$
  - $temp2 \leftarrow \text{LOAD}(d + temp1)$ ▷ $d[next[i]]$
  - $temp3 \leftarrow \text{LOAD}(d + i)$ ▷ $d[i]$
  - $temp4 \leftarrow temp3 + temp2$
  - $\text{STORE}(d + i, temp4)$
- Step 2: two PUs never have the same $next$ (injective): **exclusive**. Steps 1, 3, 5: cell i only: **exclusive**.
- Line 7 likewise: $temp5 \leftarrow \text{LOAD}(next + temp1)$, $\text{STORE}(next + i, temp5)$: exclusive.
- All loads of a line happen before its store: PU i reads the *old* values. Constant number of steps: asymptotically the same! **EREW** is sufficient.

RankComputation($next[1 \ldots N]$)

1: **forall** $i = 1, \ldots, N$ **in parallel do**
2: **if** $next[i] = \text{Nil}$ **then** $d[i] \leftarrow 0$ **else** $d[i] \leftarrow 1$
3: **while** there exists a node i such that $next[i] \neq \text{Nil}$ **do**
4: **forall** $i = 1, \ldots, N$ **in parallel do**
5: **if** $next[i] \neq \text{Nil}$ **then**
6: $d[i] \leftarrow d[i] + d[next[i]]$
7: $next[i] \leftarrow next[next[i]]$

List ranking (cont.)

How to know when to stop (line 3)? On a **CRCW** PRAM: a shared flag *done*, written in **fusion mode "and"**.

- Line 9: every PU writes ($next[i] = \text{NIL}$) into the same cell *done*: a concurrent write, merged by the "and" fusion mode. *done* is true iff every node has reached the end.
- Line 4: all PUs read *done* at the same time: a concurrent read, fine on CRCW.
- Line 9 is outside the **if**: a PU already at the end writes **true**; if only the active PUs wrote, a list where nobody is active ($N = 1$) would loop forever.
- Cost of the test: $O(1)$ per iteration. $T(N) = O(\log N)$ with N PUs.

RANKCOMPUTATIONCRCW($next[1 \dots N]$)

```
1:  forall i = 1, ..., N in parallel do
2:      if next[i] = NIL then d[i] ← 0 else 1
3:  done ← false
4:  while not done do
5:      forall i = 1, ..., N in parallel do
6:          if next[i] ≠ NIL then
7:              d[i] ← d[i] + d[next[i]]
8:              next[i] ← next[next[i]]
9:          done ← (next[i] = NIL)                ▶ and
10: return d[1 ... N]
```

List ranking (cont.)

How to know when to stop? On a **CREW/EREW** PRAM: no fusion mode, the flag must be computed by a **reduction**.

- Line 9: each PU writes its own bit $pd[i]$ (exclusive). Line 10: REDUCE with "and": $O(\log N)$.
- Line 4: all PUs read *done*: fine on CREW. On EREW, add $pdone \leftarrow \text{BCAST}(done, N)$ after line 10 ($O(\log N)$ too) and test $pdone[i]$.
- Complexity: $\lceil \log_2 N \rceil$ iterations, each $O(1)$ for the jump plus $O(\log N)$ for the test: $T(N) = O(\log^2 N)$ instead of $O(\log N)$!
- **Solution**: perform the check only every $\lceil \log_2 N \rceil$ iterations (at most $\lceil \log_2 N \rceil - 1$ useless iterations). The amortized cost of the test is $O(1)$ per iteration, and the overall complexity stays $O(\log N)$.

RANKCOMPUTATIONCREW($next[1 \ldots N]$)

```
1:  forall i = 1, ..., N in parallel do
2:     if next[i] = NIL then d[i] ← 0 else 1
3:  done ← false
4:  while not done do
5:     forall i = 1, ..., N in parallel do
6:        if next[i] ≠ NIL then
7:           d[i] ← d[i] + d[next[i]]
8:           next[i] ← next[next[i]]
9:        pd[i] ← (next[i] = NIL)
10:    done ← REDUCE(pd[1 ... N], and)
11: return  d[1 ... N]
```

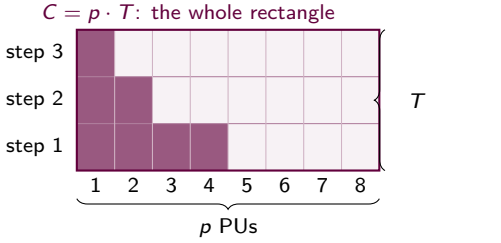
Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

Evaluating the performance of PRAM algorithms

For a problem of size n, a sequential algorithm, and a parallel algorithm on a PRAM with p PUs, we define:

- $T_{seq}(n)$: execution time of the best sequential algorithm
- $T(n,p)$: parallel execution **time** using p PUs
- $C(n,p) = p\,T(n,p)$: parallel execution **cost**: p PUs during $T(n,p)$ steps, whether they are used or not
- $W(n,p)$: **work**: total number of operations of the PUs that are *created* (by a **forall**), over all the steps



Example: REDUCE of 8 elements on a PRAM with $p = 8$ PUs: $T = 3$ steps, which create 4, 2, then 1 PUs (**forall** $i = 1, \ldots, \lceil n/2 \rceil$). $C = 8 \cdot 3 = 24$, $W = 4 + 2 + 1 = 7$: the PUs that are not created do no work, but the machine has them, so the cost pays for them anyway.

Evaluating the performance of PRAM algorithms (cont.)

- $D(n) = \lim_{p \to \infty} T(n, p)$: **depth** of the parallel algorithm
- $T_{seq}(n) \leq W(n, p) \leq C(n, p)$: parallelism potentially adds an overhead
- $S(n, p) = \dfrac{T_{seq}(n)}{T(n, p)}$: **speedup**
- $E(n, p) = \dfrac{S(n, p)}{p} = \dfrac{T_{seq}(n)}{p\, T(n, p)} = \dfrac{T_{seq}(n)}{C(n, p)}$: **efficiency**
- If $W(n, p) = O(T_{seq}(n))$, the algorithm is called **work-optimal**.
- If $C(n, p) = O(T_{seq}(n))$, the algorithm is called **efficient**.
- When the algorithm uses the maximum number of PUs for the problem (e.g., $p = n$), for simplicity we drop p and just write $T(n)$, $C(n)$, $W(n)$, $S(n)$ and $E(n)$.

Outline

1. The RAM model and sequential complexity
2. The PRAM model
3. A first PRAM algorithm: multiplying an array by 2
4. Search in an array
5. Search on an EREW PRAM, and broadcast
6. Sum of an array, and reduction
7. List ranking
8. Evaluating the performance of PRAM algorithms
9. Work and cost of the algorithms seen so far

Work and cost: MultiplyByTwo

We now evaluate every algorithm of this session on a PRAM with N PUs: $C(N) = N \cdot T(N)$, and $W(N)$ counts the operations of the PUs *created* by the **forall** loops. **Convention**: a created PU works at every step of the loop body, *including* an **if** whose condition is false (it has to evaluate it); a PU that the loop does not create (**forall** $i = 1, \ldots, N/2$) does no work, but counts in the cost: the machine has N PUs anyway.

- $T_{seq}(N) = O(N)$: one loop over the array.
- $T(N) = O(1)$: one read, one multiplication, one write per PU, all at once; $C(N) = N \cdot O(1) = O(N)$. $W(N) = O(N)$: all N PUs are created, each does $O(1)$ work.
- $W(N) = O(T_{seq}(N))$: **work-optimal**; $C(N) = O(T_{seq}(N))$: **efficient**. $S(N) = O(N)$, $E(N) = O(1)$: the ideal case.

MultiplyByTwo$(A[1 \ldots N])$

1: **forall** $i = 1, \ldots, N$ **in parallel do**
2: $\quad A[i] \leftarrow 2 \cdot A[i]$

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
MultiplyByTwo (EREW)	N	$O(N)$	$O(1)$	$O(N)$	**yes**	$O(N)$	**yes**

Work and cost: SEARCHINDEX

On a CREW PRAM (unique elements) or a CRCW PRAM (arbitrary, priority, or min/max fusion mode).

- $T_{seq}(N) = O(N)$: scan the array.
- $T(N) = O(1)$: all comparisons at once; at most one write of *idx* (CREW) or a concurrent write resolved by the mode (CRCW).
- $C(N) = O(N)$; $W(N) = O(N)$: every PU does one comparison.
- **Work-optimal** and **efficient** on both models.
- The concurrent reads of e are what forbids this time on an EREW PRAM: next slides.

SEARCHINDEX$(A[1 \dots N], e)$

```
1: idx ← 0
2: forall i = 1, ..., N in parallel do
3:     if A[i] = e then
4:         idx ← i
5: return idx
```

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
SEARCHINDEX (CREW)	N	$O(N)$	$O(1)$	$O(N)$	**yes**	$O(N)$	**yes**
SEARCHINDEX (CRCW)	N	$O(N)$	$O(1)$	$O(N)$	**yes**	$O(N)$	**yes**

Work and cost: BCAST

- $T_{seq}(N) = O(N)$; $T(N) = O(\log N)$: $\lceil \log_2 N \rceil$ iterations of an $O(1)$ body; $C(N) = O(N \log N)$.
- $W(N)$ as written: every iteration creates N PUs that all evaluate the **if**, although only 2^k copy a value: $W(N) = O(N \log N)$, **not work-optimal**.
- BCAST **can be made work-optimal**: create only the 2^k PUs that copy at iteration k (**forall** $i = 1, \ldots, \min(2^k, N - 2^k)$, no **if**): same time, $W(N) \leq 1 + 2 + 4 + \cdots < 2N = O(N)$. The version we use from now on.
- **Work-optimal** but **not efficient**: $C(N) = O(N \log N)$ pays for the PUs that are not created.

BCAST(e, N)

```
1: pe[1] ← e
2: for k = 0, ..., ⌈log2 N⌉ − 1 do
3:     forall i = 1, ..., N in parallel do
4:         if i ≤ 2^k and i + 2^k ≤ N then
5:             pe[i + 2^k] ← pe[i]
6: return pe[1 ... N]
```

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
BCAST (EREW, as written)	N	$O(N)$	$O(\log N)$	$O(N \log N)$	**no**	$O(N \log N)$	**no**
BCAST (EREW, work-optimal)	N	$O(N)$	$O(\log N)$	$O(N)$	**yes**	$O(N \log N)$	**no**

Work and cost: SearchIndexEREW

Bcast of the key, then the comparisons, on an EREW PRAM.

- $T_{seq}(N) = O(N)$.
- $T(N) = O(\log N) + O(1) = O(\log N)$: the broadcast dominates; $C(N) = O(N \log N)$.
- $W(N) = O(N)$ (work-optimal Bcast) $+ O(N)$ (one comparison per PU) $= O(N)$.
- **Work-optimal**, **not efficient**: the profile of Bcast, since the comparison adds only $O(1)$ time and $O(N)$ work.
- Versus CREW: same work, $\log N$ times more time and cost. The price of exclusive reads is paid in *time*, not in work.

SearchIndexEREW$(A[1 \dots N], e)$

1: $idx \leftarrow 0$
2: $pe[1 \dots N] \leftarrow$ Bcast(e, N)
3: **forall** $i = 1, \dots, N$ **in parallel do**
4: **if** $A[i] = pe[i]$ **then**
5: $idx \leftarrow i$
6: **return** idx

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
SearchIndexEREW	N	$O(N)$	$O(\log N)$	$O(N)$	yes	$O(N \log N)$	no

Work and cost: SumArrayCRCW

Sum of an array on a CRCW PRAM in fusion mode $+$.

- $T_{seq}(N) = O(N)$.
- $T(N) = O(1)$: one read and one concurrent write per PU; $C(N) = O(N)$, $W(N) = O(N)$.
- **Work-optimal** and **efficient**, with speedup $O(N)$.
- Too good to be true: the $N - 1$ additions are done by the memory during the concurrent write, and are not counted. This is the CRCW model being unrealistic, not a good algorithm.

SumArrayCRCW($A[1 \dots N]$)

1: $sum \leftarrow 0$
2: **forall** $i = 1, \dots, N$ **in parallel do**
3: $\quad sum \leftarrow A[i]$ ▶ fusion $+$
4: **return** sum

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
SumArrayCRCW (fusion $+$)	N	$O(N)$	$O(1)$	$O(N)$	**yes**	$O(N)$	**yes**

Work and cost: SumArrayEREW and Reduce

Sum/reduction of an array by halving, on a CREW or EREW PRAM (same analysis for both).

- $T_{seq}(N) = O(N)$: $N-1$ applications of $\otimes$.
- $T(N) = O(\log N)$: $\lceil \log_2 N \rceil$ levels of an $O(1)$ body; $C(N) = O(N \log N)$ with N PUs (half of them never created!).
- $W(N)$: $\lceil N/2 \rceil$ PUs created at the first level, then $\lceil N/4 \rceil$, ...: $W(N) = \lceil N/2 \rceil + \lceil N/4 \rceil + \cdots \leq N + \lceil \log_2 N \rceil = O(N)$.
- **Work-optimal**, **not efficient**: the mirror image of Bcast, with the same profile. $S(N) = O(N/\log N)$, $E(N) = O(1/\log N)$. Can we recover efficiency with fewer PUs? Yes: TD1, exercise 4.

Reduce$(A[1 \ldots n], \otimes)$

1: **if** $n = 1$ **then return** $A[1]$
2: **forall** $i = 1, \ldots, \lceil n/2 \rceil$ **in parallel do**
3: **if** $2i \leq n$ **then**
4: $A[i] \leftarrow A[2i-1] \otimes A[2i]$
5: **else**
6: $A[i] \leftarrow A[2i-1]$
7: **return** Reduce$(A[1 \ldots \lceil n/2 \rceil], \otimes)$

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
SumArrayEREW (CREW/EREW)	N	$O(N)$	$O(\log N)$	$O(N)$	yes	$O(N \log N)$	no
Reduce (CREW/EREW)	N	$O(N)$	$O(\log N)$	$O(N)$	yes	$O(N \log N)$	no

Work and cost: RankComputation (list ranking)

- $T_{seq}(N) = O(N)$. With N PUs: $\lceil \log_2 N \rceil$ iterations of an $O(1)$ body, plus the test of line 3: $O(1)$ per iteration on CRCW, $T(N) = O(\log N)$; a reduction per test on CREW/EREW: $O(\log^2 N)$ if tested at every iteration, $O(\log N)$ with the amortized test*.
- $W(N)$: all N PUs are created at each iteration (a node already at Nil still evaluates line 5): $W(N) = C(N) = O(N \log N)$.
- **Not work-optimal, not efficient**: a $\log N$ factor of work traded for an exponential gain in time.

RankComputation($next[1 \dots N]$)

1: **forall** $i = 1, \dots, N$ **in parallel do**
2: **if** $next[i] = \text{Nil}$ **then** $d[i] \leftarrow 0$ **else** 1
3: **while** $\exists\, i$ with $next[i] \neq \text{Nil}$ **do**
4: **forall** $i = 1, \dots, N$ **in parallel do**
5: **if** $next[i] \neq \text{Nil}$ **then**
6: $d[i] \leftarrow d[i] + d[next[i]]$
7: $next[i] \leftarrow next[next[i]]$

Algorithm	PUs	T_{seq}	T	W	Work-optimal?	C	Efficient?
RankComputation (CRCW)	N	$O(N)$	$O(\log N)$	$O(N \log N)$	**no**	$O(N \log N)$	**no**
(CREW/EREW*)	N	$O(N)$	$O(\log N)$	$O(N \log N)$	**no**	$O(N \log N)$	**no**

* termination test every $\lceil \log_2 N \rceil$ iterations (amortized); an extra $\log N$ factor on $T(N)$ and $C(N)$ otherwise.

Contact

Oguz Kaya
Université Paris-Saclay and LISN, Paris, France
oguz.kaya@universite-paris-saclay.fr
www.oguzkaya.com